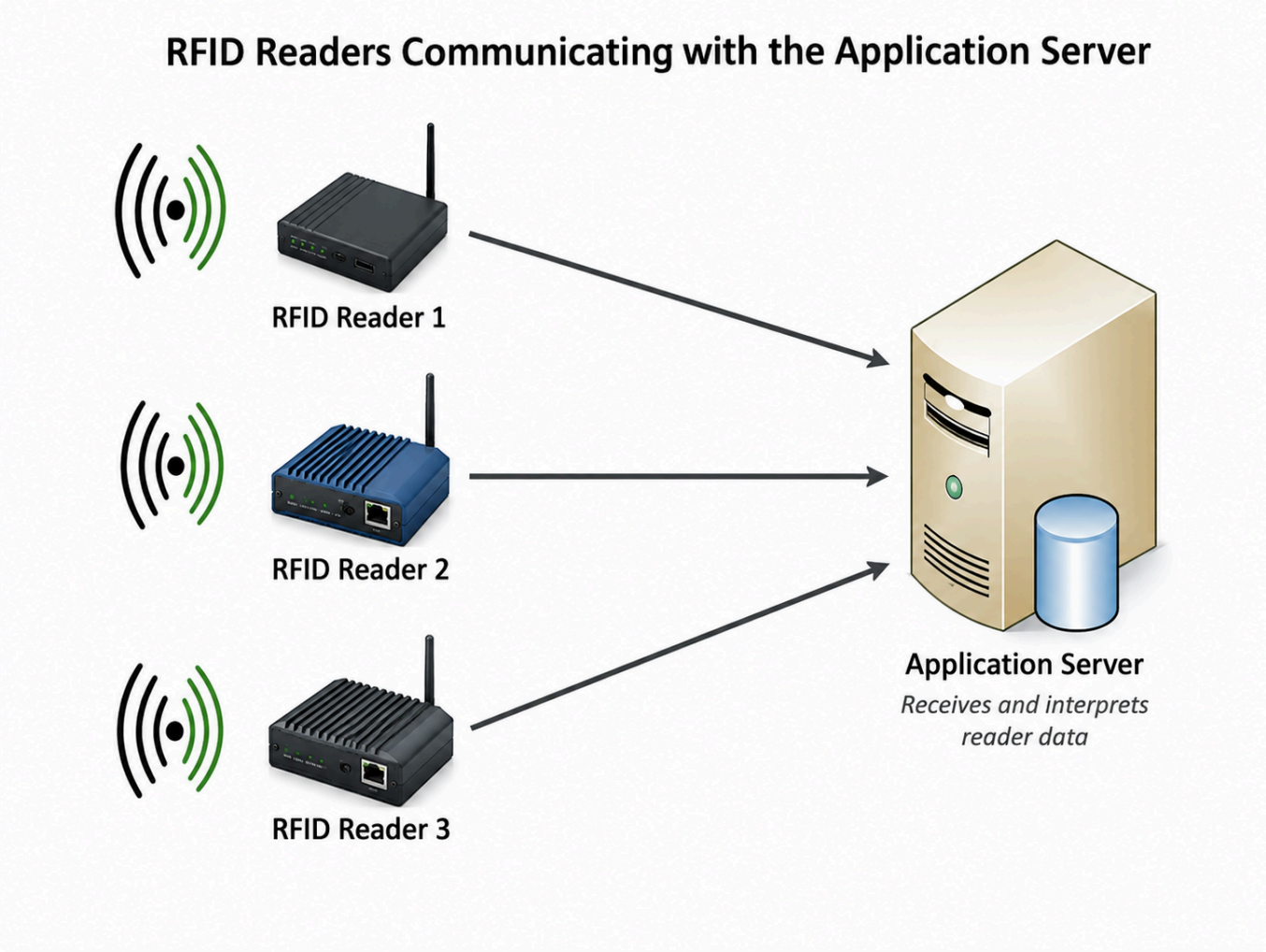
1. Objetivo

O objetivo desta etapa foi validar a comunicação em tempo real entre leitores RFID Identix EZ700 e um computador atuando como servidor local. O fluxo implementado foi:



Tag RFID → Antena → Leitor EZ700 → MQTT → Mosquitto → Ingestor Python → JSONL

O resultado principal foi positivo: dois leitores diferentes conseguiram enviar simultaneamente leituras RFID para o mesmo notebook pela rede cabeada, utilizando MQTT, sem misturar a origem dos dados.

---

2. Tecnologias utilizadas

Hardware

- Dois leitores RFID Identix EZ700.

- Antenas RFID UHF.
- Tags RFID UHF passivas.
- Fontes/injetores PoE160S.
- Cabos Ethernet.
- Notebook Ubuntu funcionando como servidor local.

Software e protocolos

- Ubuntu Linux.
- Docker e Docker Compose.
- Eclipse Mosquitto `2.1.2-alpine`.
- MQTT pela porta TCP 1883.
- Python e ambiente virtual `.venv`.
- Ingestor `rfid_ingestion`.
- Armazenamento em JSON Lines, arquivo `.jsonl`.
- Ferramentas de diagnóstico:
  - `ping`
  - `ip route`
  - `ip -br addr`
  - `ss`
  - `nc`
  - `mosquitto_sub`

---

3. O que são MQTT e Mosquitto

MQTT

MQTT é um protocolo de comunicação baseado no modelo de publicação e assinatura.

No nosso caso:

- Os leitores EZ700 são os publicadores.
- O Mosquitto recebe as mensagens.
- O ingestor Python é o assinante.
- Os tópicos identificam a origem das leituras.

Exemplo de mensagem enviada pelo leitor:

{

"epc": "3034000000000C4000000072",

"rssi": -6350,

"antenna": 1,

"timestamp": "1821775"

}

O payload contém:

- `epc`: identificação da tag.
- `rssi`: intensidade do sinal recebido.
- `antenna`: antena responsável pela leitura.
- `timestamp`: contador ou marca temporal produzida pelo leitor.

Mosquitto

Mosquitto é o software que implementa o broker MQTT.

O broker:

- Recebe as mensagens dos leitores.
- Identifica o tópico de cada mensagem.
- Repassa as mensagens aos clientes inscritos.
- Não é o responsável principal por salvar os dados no arquivo.

Quem salva os eventos é o ingestor Python.

A porta utilizada foi:

TCP 1883

Essa é a porta MQTT sem criptografia. No laboratório foi suficiente, mas uma instalação definitiva no hospital deverá avaliar autenticação, regras de firewall e, possivelmente, MQTT com TLS.

QoS e retenção

As mensagens foram observadas com:

- QoS 1: entrega pelo menos uma vez.
- `retain: false`: o broker não mantém a última leitura como mensagem retida.

O QoS 1 oferece maior confiabilidade, mas permite mensagens repetidas. Além disso, o próprio processo de inventário RFID lê a mesma tag várias vezes por segundo. Portanto, a futura aplicação precisará aplicar deduplicação ou agregação.

---

4. Configuração do notebook como servidor

O notebook estava conectado à rede cabeada do Instituto de Computação com:

IP: 172.20.109.132

Máscara: 255.255.0.0 ou /16

Gateway: 172.20.7.1

O endereço de rede era:

172.20.0.0/16

Os leitores receberam endereços na mesma rede:

Equipamento	Endereço
Notebook/broker	`172.20.109.132`
Leitor 1	`172.20.109.120`
Leitor 2	`172.20.109.184`

Como todos estavam na mesma sub-rede `/16`, eles conseguiam conversar diretamente, sem precisar passar pelo gateway.

Inicialização do broker

No projeto:

cd ~/Code/rfid-hospital-mvp

docker compose up -d

O Docker Compose iniciou um contêiner com o Mosquitto e publicou a porta 1883 do contêiner em todas as interfaces do notebook:

0.0.0.0:1883

Isso foi importante porque, se o Mosquitto escutasse somente em `127.0.0.1`, apenas aplicações do próprio notebook conseguiriam acessá-lo.

Verificação da porta

sudo ss -lntp | grep ':1883'

Resultado esperado:

0.0.0.0:1883

Também foi realizado o teste:

nc -vz 172.20.109.132 1883

Resultado esperado:

Connection succeeded

Verificação direta das mensagens MQTT

Para visualizar mensagens dos dois leitores:

mosquitto_sub -h 127.0.0.1 -p 1883 -t 'rfid/+/tags' -v

Para visualizar somente o leitor 1:

mosquitto_sub -h 127.0.0.1 -p 1883 -t 'rfid/EZ700-82-8A-54/#' -v

Para visualizar somente o leitor 2:

mosquitto_sub -h 127.0.0.1 -p 1883 -t 'rfid/EZ700-82-89-14/#' -v

Inicialização do ingestor

cd ~/Code/rfid-hospital-mvp

source .venv/bin/activate

python -m rfid_ingestion

O ingestor se inscreveu no padrão:

rfid/+/tags

O símbolo `+` representa qualquer identificador de leitor. Dessa forma, um único ingestor consegue receber dados de vários leitores.

Os eventos foram armazenados em:

data/raw_events.jsonl

Cada linha contém um objeto JSON completo, incluindo:

- Horário em que o notebook recebeu a mensagem.
- Tópico MQTT.
- QoS.
- Payload original.
- Payload convertido para JSON.
- Informação sobre possível erro de conversão.

Exemplo:

{

"received_at_utc": "2026-08-20T15:42:17.301Z",

"topic": "rfid/EZ700-82-8A-54/tags",

"qos": 1,

"retain": false,

"payload_json": {


```
"epc": "30340000000000C4000000072",

"rssi": -6350,

"antenna": 1,

"timestamp": "1821775"
```


},

"parse_error": null

}

---

5. Separação entre os leitores

Os dados de diferentes leitores chegam ao mesmo broker e à mesma porta, mas são separados pelos tópicos MQTT.

Leitor	Client ID	Tópico de leitura
Leitor 1	`EZ700-82-8A-54`	`rfid/EZ700-82-8A-54/tags`
Leitor 2	`EZ700-82-89-14`	`rfid/EZ700-82-89-14/tags`

O payload enviado pelo EZ700 não contém diretamente o identificador do leitor. A origem é obtida pelo campo `topic` salvo pelo ingestor.

É indispensável que cada leitor tenha:

- Client ID exclusivo.
- Conjunto de tópicos exclusivo.

Se dois leitores utilizarem o mesmo Client ID, um poderá desconectar o outro do broker.

---

6. Testes realizados

Teste 1 — Um leitor conectado ao notebook

Foi conectado o primeiro EZ700 à rede cabeada do IC.

Foram validados:

- Recebimento de IP por DHCP.
- Comunicação por `ping`.
- Acesso ao firmware pelo IP Ethernet.
- Conexão do leitor ao Mosquitto.
- Recebimento das mensagens com `mosquitto_sub`.
- Recebimento pelo ingestor.
- Conversão correta do payload para JSON.
- Armazenamento em `raw_events.jsonl`.

Resultado: aprovado.

Teste 2 — Dois leitores simultâneos

O segundo leitor foi conectado à mesma rede cabeada.

Foram validados:

- IP independente para cada leitor.

- Client IDs diferentes.
- Tópicos MQTT diferentes.
- Publicação simultânea na porta 1883.
- Recebimento dos dois leitores pelo mesmo ingestor.
- Identificação da origem pelo tópico.
- Armazenamento cronológico no mesmo JSONL.

Resultado: aprovado.

Teste 3 — Simulação de deslocamento

Os leitores foram posicionados como dois pontos ou setores diferentes.

Uma tag foi movimentada entre as áreas de leitura para simular o deslocamento de uma pessoa ou equipamento pelo hospital.

Foram produzidos dados contendo:

- EPC da tag.
- Leitor responsável pela leitura.
- RSSI.
- Antena.
- Ordem temporal de recebimento.

Esse teste poderá ser analisado posteriormente para estudar:

- Sequência leitor 1 → leitor 2.
- Sequência inversa.
- Intensidade de sinal em cada ponto.
- Sobreposição entre zonas de leitura.
- Possível inferência de localização e movimentação.

Resultado da coleta: concluído, com sessão salva para análise.

---

7. Testes teoricamente viáveis com alta probabilidade de sucesso

Notebook e leitores no mesmo Wi-Fi

Existe alta probabilidade de funcionar porque, após os dispositivos receberem endereços IP e conseguirem alcançar o broker, o funcionamento do MQTT é praticamente igual ao da rede cabeada.

O meio muda de Ethernet para Wi-Fi, mas o fluxo permanece:

Leitor → IP do notebook → porta 1883 → Mosquitto

Para funcionar, é necessário:

- O leitor ser compatível com a segurança da rede.
- O DHCP entregar um endereço IP.
- Não haver isolamento entre clientes Wi-Fi.
- A porta 1883 não estar bloqueada.
- Os dispositivos conseguirem se alcançar.

Notebook cabeado e leitores no Wi-Fi

Também possui alta probabilidade de funcionar se a infraestrutura permitir comunicação entre a rede cabeada e o Wi-Fi.

Os dispositivos não precisam estar exatamente no mesmo meio físico. Eles precisam ter:

- Rota entre as redes.
- Permissão de firewall.
- Acesso ao IP do broker na porta 1883.

Visualização no sistema em tempo real

Como as mensagens já chegam em tempo real ao ingestor, existe alta probabilidade de o futuro dashboard funcionar corretamente.

O software precisará consumir:

- Diretamente o MQTT; ou
- Uma camada intermediária que processe e armazene os eventos.

O teste do dashboard deverá verificar atualização em tempo real, deduplicação e cálculo de localização.

Implantação no hospital

A arquitetura deverá funcionar no HU após:

- Conectar os leitores à rede disponível.

- Descobrir o novo IP do notebook ou servidor.
- Atualizar esse IP no campo MQTT de cada leitor.
- Liberar a porta necessária.
- Verificar as regras de comunicação da rede hospitalar.

É recomendável utilizar IP fixo ou reserva DHCP para o servidor. Assim, o endereço do broker não mudará constantemente.

---

8. Testes pendentes

Ainda precisam ser testados:

- Todos os dispositivos conectados ao mesmo Wi-Fi.
- Leitores no Wi-Fi e notebook na rede cabeada.
- Dispositivos em sub-redes diferentes.
- Comunicação com firewall ou regras de rede do HU.
- Instalação real no hospital.
- Atualização do dashboard em tempo real.
- Reconexão após queda de energia ou rede.
- Teste de longa duração.
- Perda de pacotes e estabilidade.
- Vários leitores durante períodos prolongados.
- Deduplicação das leituras.
- Cálculo de localização por RSSI, sequência e zona.
- Uso de autenticação MQTT.
- Avaliação de TLS para proteger a comunicação.
- Rotação dos arquivos de eventos por teste, hora ou dia.

O hotspot do celular não pôde ser validado porque o sinal estava insuficiente no laboratório.

O Wi-Fi do IC também não pôde ser validado devido ao erro de obtenção de endereço IP, porém é um teste pendente a se fazer para tentar validar o funcionamento com os dispositivos em redes diferentes.

---

9. Passo a passo para configurar um leitor EZ700

9.1 Preparação física

Com o leitor desligado:

1. Conectar a antena RFID na porta configurada, normalmente a antena 1.
2. Conferir se o conector está firme.
3. Não conectar ou desconectar a antena com o leitor energizado.
4. Conectar o cabo que vai para o leitor na porta:

PWR + DATA OUT

5. Conectar o cabo da rede do local na porta:

DATA IN

6. Energizar a fonte PoE.

Ligação:

Rede do local → DATA IN → PoE160S → PWR+DATA OUT → EZ700

9.2 Acessar inicialmente o firmware pela rede do leitor

O EZ700 cria sua própria rede Wi-Fi para configuração.

1. Procurar no notebook uma rede com nome semelhante a:

EZ700 82:8A:54

2. Conectar o notebook a essa rede.
3. Verificar o endereço recebido:

ip -br addr

ip route

No laboratório, o notebook recebeu:

192.168.4.2/24

O leitor estava no gateway:

192.168.4.1

4. Abrir no navegador:

http://192.168.4.1

Esse foi o endereço utilizado no laboratório. Caso não funcione em outra unidade, verificar o gateway mostrado pelo `ip route`.

9.3 Verificar a configuração regional

No Dashboard, verificar:

- Modelo: EZ700.
- Firmware.
- Status RFID.
- Região RAIN RFID.

No Brasil, confirmar que a região está configurada como:

BRAZIL

Também verificar se:

RFID status: Inventory running

9.4 Conectar o leitor à rede que será utilizada

Existem duas opções.

Opção A — Ethernet

1. Ligar a rede do local no `DATA IN` do PoE.
2. Manter a configuração Ethernet em DHCP, quando disponível.
3. Aguardar o leitor receber um endereço.
4. Consultar o Dashboard.

O resultado esperado é semelhante a:

Ethernet: Connected

IP address: endereço fornecido pela rede

Netmask: máscara da rede

Gateway: gateway da rede

Se o endereço permanecer `0.0.0.0`, o leitor não recebeu IP.

Opção B — Wi-Fi

Na página de rede do firmware:

1. Acessar a configuração `WLAN client`.
2. Informar o SSID da rede.
3. Informar a senha.
4. Selecionar o padrão de segurança correto, por exemplo:

WPA2-PSK

5. Selecionar:

DHCP (automatic)

6. Clicar em `Apply`.
7. Aguardar entre 30 e 90 segundos.

É normal a página cair temporariamente porque o leitor reinicia ou altera sua interface de rede.

Depois do reinício, o Dashboard deverá mostrar:

- SSID.
- RSSI.
- Canal.
- Endereço IP.
- Máscara.
- Gateway.

A exibição do SSID e do RSSI sem um endereço IP válido não é suficiente. O leitor precisa deixar de mostrar `0.0.0.0`.

9.5 Descobrir o IP recebido pelo leitor

Ainda conectado à rede de configuração do leitor, verificar no Dashboard qual endereço foi atribuído à interface Ethernet ou WLAN.

Também é possível obter o endereço por:

- Servidor DHCP.
- Painel do roteador.
- Administrador da rede.
- Ferramentas de descoberta autorizadas.

Anotar o IP do leitor.

Exemplo:

172.20.109.184

9.6 Sair da rede própria do leitor

Depois que o EZ700 estiver conectado à rede desejada:

1. Desconectar o notebook da rede Wi-Fi criada pelo EZ700.
2. Conectar o notebook à mesma rede do leitor ou a uma rede com rota até ele.
3. Testar:

ping -c 4 IP_DO_LEITOR

Exemplo:

ping -c 4 172.20.109.184

4. Abrir o firmware pelo novo endereço:

http://IP_DO_LEITOR

Exemplo:

http://172.20.109.184

A partir desse momento, a configuração é feita pelo endereço fornecido pela infraestrutura local.

9.7 Preparar o servidor MQTT

No notebook:

cd ~/Code/rfid-hospital-mvp

docker compose up -d

Descobrir o IP do notebook na rede utilizada:

ip -br addr

ip route

Verificar a porta:

sudo ss -lntp | grep ':1883'

Testar:

nc -vz IP_DO_NOTEBOOK 1883

O Mosquitto deve aparecer escutando em:

0.0.0.0:1883

9.8 Configurar o MQTT no leitor

No firmware do EZ700:

1. Acessar `Data Output`.
2. Abrir a seção `MQTT`.
3. Marcar `Enable`.
4. Preencher a URL com o IP do notebook/servidor:

mqtt://IP_DO_SERVIDOR

Exemplo utilizado:

mqtt://172.20.109.132

5. Preencher a porta:

1883

6. Criar um Client ID exclusivo.

Exemplo:

EZ700-82-89-14

7. No ambiente de laboratório, deixar usuário e senha vazios.
8. Selecionar saída em JSON.
9. Ativar heartbeat.
10. Configurar os tópicos.

Exemplo para o segundo leitor:

Inventory topic: rfid/EZ700-82-89-14/tags

Heartbeat topic: rfid/EZ700-82-89-14/heartbeat

Command topic: rfid/EZ700-82-89-14/command

11. Clicar em `Apply`.
12. Aguardar o leitor aplicar a configuração.
13. Recarregar ou reabrir a página pelo IP do leitor.
14. Verificar no Dashboard:

MQTT: connected

O endereço informado na URL deve ser o IP do servidor, e não o IP do próprio leitor.

9.9 Validar a publicação

No notebook:

mosquitto_sub -h 127.0.0.1 -p 1883 -t 'rfid/+/tags' -v

Aproximar uma tag da antena.

O resultado esperado será semelhante a:

rfid/EZ700-82-89-14/tags {"epc":"...","rssi":-6200,"antenna":1,"timestamp":"..."}

Depois, iniciar o ingestor:

cd ~/Code/rfid-hospital-mvp

source .venv/bin/activate

python -m rfid_ingestion

Para não perder as primeiras mensagens, a ordem recomendada é:

1. Subir o Mosquitto.
2. Iniciar o ingestor.
3. Ligar os leitores.
4. Iniciar o percurso com as tags.

---

10. Problemas encontrados

Wi-Fi do IC sem fornecer endereço

O leitor encontrou o `IC-ALUNOS` e mostrou:

- SSID.
- RSSI.
- Canal.

Entretanto, permaneceu com:

IP: 0.0.0.0

O notebook também apresentou mensagens como:

Não foi possível obter endereço IP

ou:

Falha na conexão da rede

A causa provável foi instabilidade ou falha no DHCP da rede do IC. Esse problema aconteceu antes da camada MQTT e, portanto, não invalida o funcionamento comprovado pela rede cabeada.

Queda da página após aplicar configurações

Após clicar em `Apply`, a página do leitor desapareceu temporariamente.

Isso ocorreu porque o dispositivo reiniciou ou reaplicou suas interfaces. O funcionamento retornou após aguardar aproximadamente um minuto.

Conflito entre duas instalações do Mosquitto

Existiam dois Mosquittos no notebook:

- Um instalado por Snap e iniciado automaticamente.
- Outro executado pelo Docker Compose.

O Mosquitto do Snap ocupava:

127.0.0.1:1883

Isso impedia o Docker de publicar:

0.0.0.0:1883

O processo foi identificado por:

ps -o pid,ppid,user,lstart,cmd -p PID

sudo readlink -f /proc/PID/exe

sudo cat /proc/PID/cgroup

Foi identificado o serviço:

snap.mosquitto.mosquitto.service

A solução foi parar e desabilitar o Mosquitto do Snap e recriar o contêiner Docker.

Depois da correção:

0.0.0.0:1883 → funcionando

Leitor reiniciando e apitando repetidamente

O segundo leitor apitava várias vezes e não detectava tags.

A causa foi um problema na fonte ou no contato da alimentação PoE. Depois de corrigir a fonte/conexão, o leitor voltou a iniciar normalmente e detectou as tags.

Esse diagnóstico foi feito observando:

- Repetição do apito de inicialização.
- Instabilidade do dispositivo.
- Falta de leitura.
- Possível reinicialização do tempo de atividade.

Rotas diferentes no notebook

O notebook estava simultaneamente:

- Na rede cabeada do IC.
- Na rede Wi-Fi criada pelo leitor.

Isso gerou duas rotas padrão e dois endereços diferentes. Foi necessário observar cuidadosamente qual interface era usada para acessar o leitor e qual interface era usada para o servidor.

IP do servidor sujeito a alteração

O endereço `172.20.109.132` foi fornecido por DHCP e pertence apenas à rede utilizada no laboratório.

Quando o notebook for levado ao HU, ele receberá outro endereço. Será necessário:

1. Descobrir o novo IP.
2. Acessar cada leitor.
3. Atualizar a URL do broker MQTT.
4. Confirmar novamente `MQTT: connected`.

A solução mais adequada para implantação é solicitar IP fixo ou reserva DHCP para o servidor.

Alto volume de leituras repetidas

O EZ700 realiza inventário contínuo e pode ler a mesma tag muitas vezes por segundo.

Isso não representa necessariamente erro. Os dados brutos foram preservados, mas o software deverá posteriormente aplicar:

- Janela temporal.
- Deduplicação.
- Agrupamento por EPC.
- Comparação de RSSI.
- Inferência de mudança de zona.

---

11. Conclusão atual

A etapa foi bem-sucedida porque comprovou o funcionamento do caminho técnico principal:

Dois EZ700 → rede cabeada → MQTT → Mosquitto → ingestor → JSONL

O teste por Wi-Fi não apresentou falha no MQTT. Ele não pôde ser executado porque a rede do IC não forneceu endereço IP de maneira estável.

Portanto, a arquitetura de comunicação e ingestão está validada. As próximas validações devem se concentrar na infraestrutura do hospital, nas regras de rede, no dashboard em tempo real e na lógica de interpretação das leituras.

12. Imagens do firmware

Configured data outputs

Service	Details	Enabled	Connected
TCP socket server	192.168.4.1 | 14150	yes	
Websocket server	192.168.4.1 | 8080	yes	
MQTT broker	mqtt://ittsh.synergy.com.br | 1883	no	
USB		no	

TCP socket server

MQTT

USB

TCP socket server

TCP socket server settings

Enable: ☑

TCP socket server port: 14150

TCP socket server parameters:
- **keepAlive**: Enabled (1)
- **keepIdle**: 5 seconds
- **keepInterval**: 5 seconds
- **keepCount**: 5 attempts

Output format

JSON: ◉ Custom: ○

ATTENTION! The Decoded GS1 barcode, EAN/UPC and EPC serial fields will be reported only if the "Decode GS1 tag EPC memory data" option is enabled on the "RAIN RFID / Output Data Formatting and GS1 Epc decoding" configuration section.

Heartbeat

Enable: ☑

Apply

MQTT

USB

MQTT

MQTT broker settings

Enable: ☐

Url: mqtt://ittsh.synergy.com.br

Port: 1883

Client ID: device-test

Username: device_test

Password: •••••••• ☑ Show

Inventory's topic: clientid/inventory

Heartbeat's topic: clientid/heartbeat

Command's topic: clientid/command

Output format

JSON: ◉ Custom: ○

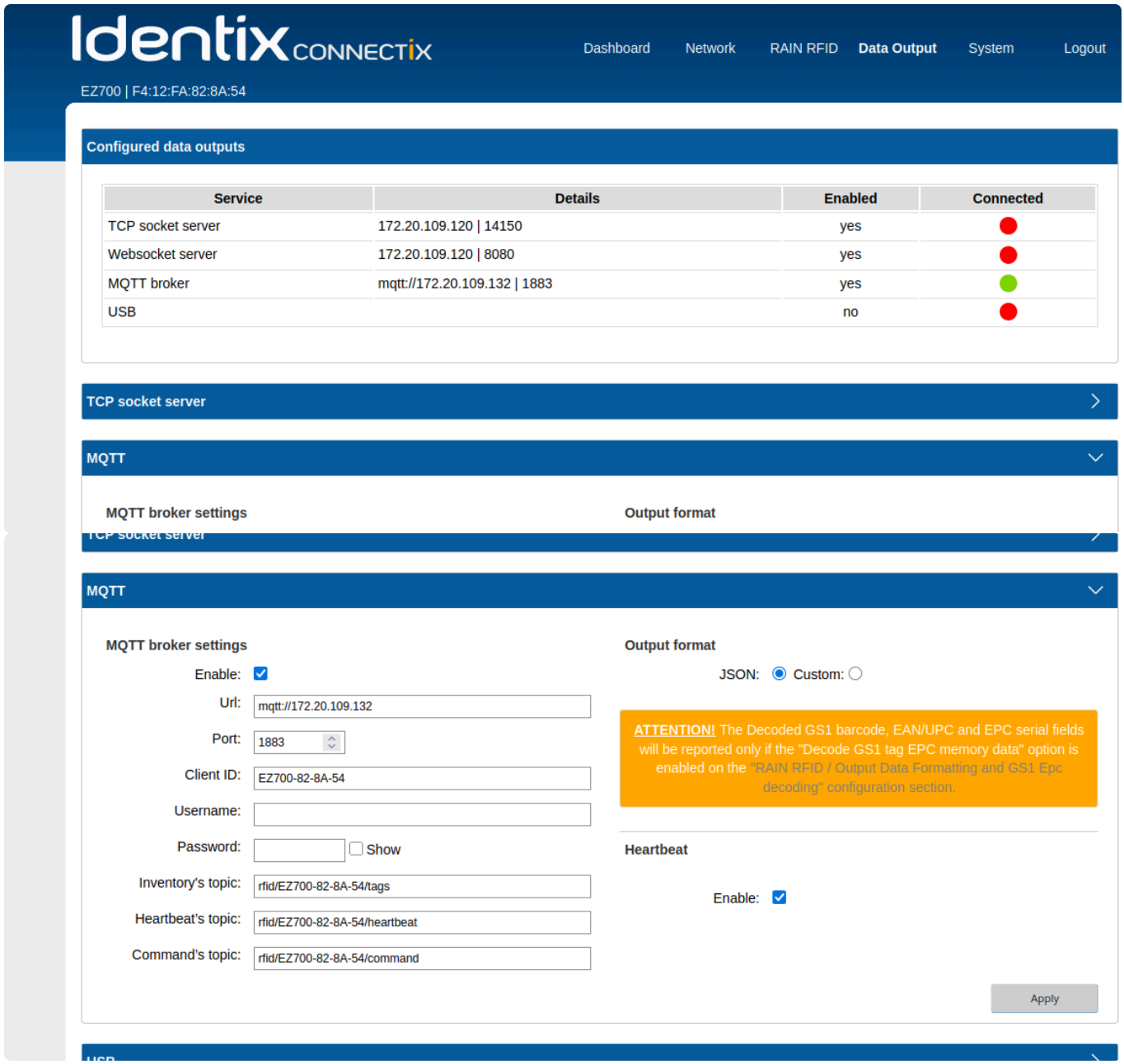
ATTENTION! The Decoded GS1 barcode, EAN/UPC and EPC serial fields will be reported only if the "Decode GS1 tag EPC memory data" option is enabled on the "RAIN RFID / Output Data Formatting and GS1 Epc decoding" configuration section.

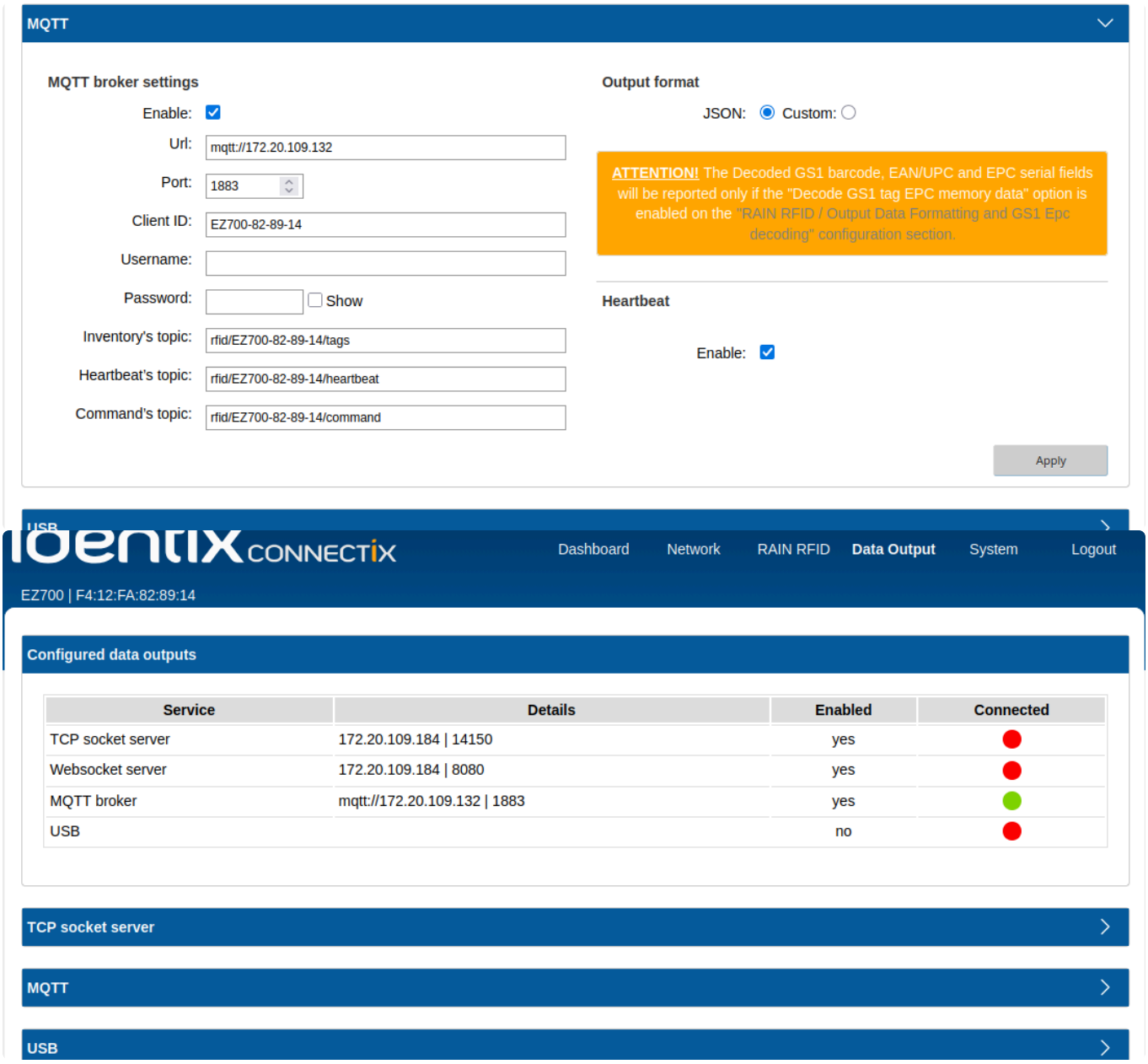
Heartbeat

Enable: ☑

Apply

USB


Identix CONNECTIX
Dashboard
Network
RAIN RFID
Data Output
System
Logout
EZ700 | F4:12:FA:82:8A:54
Configured data outputs
Service
Details
Enabled
Connected
TCP socket server
172.20.109.120 | 14150
yes
Websocket server
172.20.109.120 | 8080
yes
MQTT broker
mqtt://172.20.109.132 | 1883
yes
USB
no
TCP socket server
MQTT
MQTT broker settings
Output format
MQTT
MQTT broker settings
Enable:
Url:
mqtt://172.20.109.132
Port:
1883
Client ID:
EZ700-82-8A-54
Username:
Password:
Show
Inventory's topic:
rfid/EZ700-82-8A-54/tags
Heartbeat's topic:
rfid/EZ700-82-8A-54/heartbeat
Command's topic:
rfid/EZ700-82-8A-54/command
Output format
JSON:
Custom:
ATTENTION! The Decoded GS1 barcode, EAN/UPC and EPC serial fields will be reported only if the "Decode GS1 tag EPC memory data" option is enabled on the "RAIN RFID / Output Data Formatting and GS1 Epc decoding" configuration section.
Heartbeat
Enable:
Apply


MQTT
MQTT broker settings
Enable:
Url:
mqtt://172.20.109.132
Port:
1883
Client ID:
EZ700-82-89-14
Username:
Password:
Show
Inventory's topic:
rfid/EZ700-82-89-14/tags
Heartbeat's topic:
rfid/EZ700-82-89-14/heartbeat
Command's topic:
rfid/EZ700-82-89-14/command
Output format
JSON:
Custom:
ATTENTION! The Decoded GS1 barcode, EAN/UPC and EPC serial fields will be reported only if the "Decode GS1 tag EPC memory data" option is enabled on the "RAIN RFID / Output Data Formatting and GS1 Epc decoding" configuration section.
Heartbeat
Enable:
Apply

USB
identix CONNECTIX
Dashboard
Network
RAIN RFID
Data Output
System
Logout
EZ700 | F4:12:FA:82:89:14
Configured data outputs
Service
Details
Enabled
Connected
TCP socket server
172.20.109.184 | 14150
yes
Websocket server
172.20.109.184 | 8080
yes
MQTT broker
mqtt://172.20.109.132 | 1883
yes
USB
no
TCP socket server
MQTT
USB